ТОО ”Колледж Хекслет”

Лабораторная работа №2

Хеширование и безопасное хранение паролей

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_4. Хеширование и безопасное хранение паролей.

Цель: Реализовать безопасное хеширование и проверку паролей через Argon2, интегрировать passwordService в регистрацию и вход пользователя и убедиться, что исходный password не сохраняется и не возвращается клиенту.

Условие: RO4_4/lesson2/ЛР-2.pdf

Требования задания

• Для password hashing используется пакет argon2.

• Криптографическая логика находится в passwordService.js.

• Регистрация использует hashPassword().

• Login использует verifyPassword().

• В PostgreSQL хранится passwordHash, а не исходный password.

• Два одинаковых passwords создают разные encoded hashes.

• Правильный password успешно проходит verify.

• Неправильный password не проходит verify.

• passwordHash отсутствует в HTTP-ответах.

• Студент может объяснить назначение salt и memory-hard hashing.

Ход выполнения

Криптографические операции сосредоточены в passwordService. Argon2id создаёт encoded hash со случайной salt, поэтому одинаковые пароли дают разные строки. verify() учитывает параметры и salt из encoded hash. Простое сравнение двух hash через === не подходит. hash-benchmark.js измеряет время hash/verify без вывода исходного пароля или его хеша. Быстрый SHA-256 не заменяет memory-hard password hashing.

Проект: RO4_4/lesson2

Запуск: npm install; npm start. Для PostgreSQL параметры подключения задаются в .env по образцу .env.example. Порядок применения SQL и миграций приведён в README.md проекта.

services/passwordService.js

services/passwordService.js

```
import * as argon2 from 'argon2';
export function hashPassword(password) { return argon2.hash(password); }
export function verifyPassword(hash, password) { return argon2.verify(hash, password); }
```

Листинг 1. services/passwordService.js

services/authService.js

services/authService.js

```
import { prisma } from '../config/prisma.js';
import { AppError } from '../errors/AppError.js';
import { hashPassword, verifyPassword } from './passwordService.js';
const safeUser = { id: true, name: true, email: true, role: true };
export function findUserById(id) { return prisma.user.findUnique({ where: { id },
select: safeUser }); }
export async function registerUser({ name, email, password }) {
    email = email.trim().toLowerCase();
    if (await prisma.user.findUnique({ where: { email } })) throw new AppError('Email
already registered', 409);
    const passwordHash = await hashPassword(password);
    try { return await prisma.user.create({ data: { name: name.trim(), email,
passwordHash }, select: safeUser }); }
    catch (error) { if (error.code === 'P2002') throw new AppError('Email already
registered', 409); throw error; }
}
export async function authenticateUser(email, password) {
    const user = await prisma.user.findUnique({ where: { email:
email.trim().toLowerCase() } });
    if (!user || !user.passwordHash.startsWith('$argon2') || !await
verifyPassword(user.passwordHash, password)) return null;
    return { id: user.id, name: user.name, email: user.email, role: user.role };
}
```

Листинг 2. services/authService.js

hash-benchmark.js

hash-benchmark.js

```
import { hashPassword, verifyPassword } from './services/passwordService.js';
const password = 'Example123!';
console.time('argon2.hash');
const first = await hashPassword(password);
console.timeEnd('argon2.hash');
const second = await hashPassword(password);
console.log('Different hashes:', first !== second);
console.time('argon2.verify');
console.log('Correct:', await verifyPassword(first, password));
console.timeEnd('argon2.verify');
console.log('Wrong:', await verifyPassword(first, 'wrong-password'));
```

Листинг 3. hash-benchmark.js

Проверка результата

Проверено: 05.11.2026 13:38 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Реальный PostgreSQL

• HTTP endpoints

• CRUD сохраняется в PostgreSQL

• UNIQUE и FOREIGN KEY; категории перенесены без потери товаров

• Связи 1:1, 1:N, M:N; CHECK(quantity > 0); составной ключ

• include/select, relation filter some, историческая сумма заказа

• Миграции description/slug применены; повторный seed без дублей

• Атомарный заказ COMMIT/ROLLBACK; цена из БД; валидация позиций

• Два зарегистрированных аккаунта с одним паролем имеют разные хеши; оба проходят login

• Регистрация/login; одинаковая ошибка для неверных credentials

• Argon2id, разные salt, пароль и хеш отсутствуют в ответах

• Chromium: login через fetch, сообщение при неверном пароле, имя после входа, очистка поля пароля

• Prisma Studio открыт в Chromium; таблица products содержит Keyboard

• Chromium: login через fetch, сообщение при неверном пароле, имя после входа, очистка поля пароля

HTTP-запросы

GET /api/products → 200

GET /api/products/1 → 200

GET /api/products/999999 → 404

GET /api/products/abc → 400

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?minPrice=abc → 400

GET /api/products?inStock=maybe → 400

GET /api/products?minPrice=500&maxPrice=100 → 400

POST /api/products → 403

POST /api/products → 400

POST /api/products → 201

PATCH /api/products/6 → 200

DELETE /api/products/6 → 200

GET /api/products/6 → 404

POST /api/products → 409

GET /api/orders/1 → 200

GET /api/orders?productId=1 → 200

GET /api/orders/999999 → 404

POST /api/products → 201

DELETE /api/products/8 → 200

POST /api/orders → 201

POST /api/orders → 409

POST /api/orders → 400

POST /api/orders → 400

POST /api/orders → 400

POST /api/orders → 404

POST /api/auth/register → 201

POST /api/auth/register → 409

POST /api/auth/register → 400

POST /api/auth/register → 400

POST /api/auth/register → 400

POST /api/auth/login → 200

POST /api/auth/login → 401

POST /api/auth/login → 401

POST /api/auth/register → 201

POST /api/auth/login → 200

GET /api/unknown → 404

Примеры ответов

POST /api/auth/register → 201

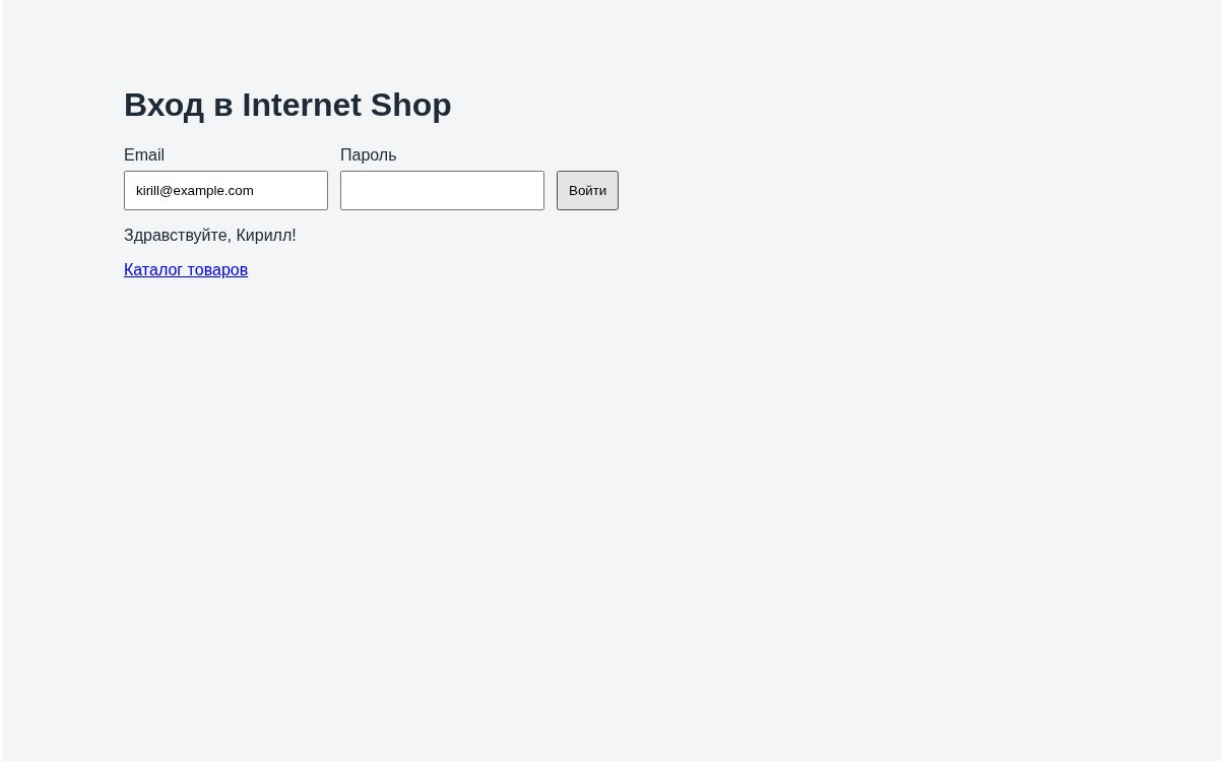

```
{
  "user": {
    "id": 4,
    "name": "Кирилл",
    "email": "kirill@example.com",
    "role": "user"
```

```
  }
}
```


POST /api/auth/login → 200

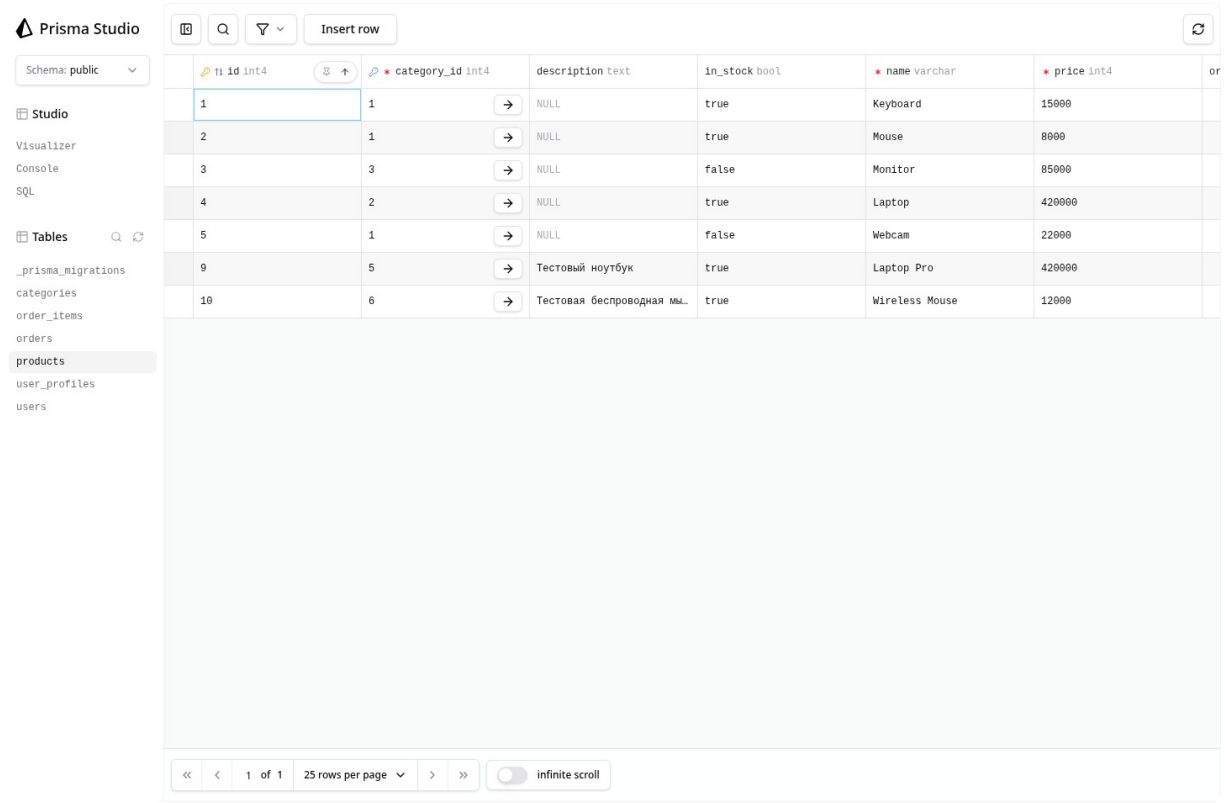

```
{
  "user": {
    "id": 4,
    "name": "Кирилл",
    "email": "kirill@example.com",
    "role": "user"
  }
}
```

Проверка в браузере Chromium



Скриншот работающего приложения после выполнения действий через интерфейс.

Проверка данных в Prisma Studio



Prisma Studio: таблицы и данные проверенной учебной базы.

Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.